Systemudvikling og softwarearkitektur

Marts 2026

Fornavn Efternavn **Fornavn Efternavn** **Fornavn Efternavn**

Teknisk Rapport
Datamatiker uddannelsen
25. March 2026

Formalia
Antal anslag: 13863
Antal normalsider: 5.8

Indhold

1 Opstart og Projektstrategi . . . 1
1.1 Det kunstige problemdomæne . . . 1
1.2 Opgaveformulering vs. agil metode . . . 1
1.3 Iterationsstruktur . . . 2
1.4 Slices . . . 2
2 Planlægning og tidsestimering . . . 3
2.1 Use Case Workshop . . . 3
2.2 Value- og risikodrevet . . . 3
3 Forløbet . . . 4
3.1 Faktiske iterationsstruktur . . . 4
4 Arkitektur og design . . . 6
4.1 Composite Pattern . . . 6
4.2 Facade Pattern . . . 7
5 Diskussion . . . 8
5.1 Andre Patterns . . . 8
5.2 SOLID og GRASP . . . 8
6 Refleksion . . . 9
6.1 Agil metode i et læringsscenarie . . . 9
6.2 Task-formuleringer . . . 9
7 Litteraturliste . . . 10
8 Bilag . . . 11

1 Opstart og Projektstrategi

1.1 Det kunstige problemdomæne

Vores projekt har fokuseret på en case bestående af *Bostedet Tulipanvej* der huser mentalt udfordrede borgere. Derudover tilbyder stedet bostøtte, hvilket er en støtteordning som borgerne modtager i hjemmet, som en overgang til at kunne bo selvstændigt i egen bolig. Vores domænekendskab bygger på en privat samtale vi havde med en specialpædagog, Laura, der arbejder i bostøtten. Ud fra den lærte vi at pædagogerne har ansvaret for at koordinere og planlægge bogerbesøgene og hvem der bruger deres begrænsede antal biler hvornår. Denne ad hoc koordinering er kritisk for arbejdsgangen, da de supportere borgere i hele Silkeborg kommune. Senere modtog vi en SMS fra Laura (Afsnit Bilag A -), hvorfra vi kunne udlede:

- 14 pædagoger
- 4 cykler delt med botilbuddet
- 4 biler kun til bostøtten
- Booking (af biler) sker i Outlook
- Der er retningslinjer for, hvor langt frem man må booke

På trods af denne information måtte vi erkende at vores kendskab til domænet var for begrænset til at vi kunne gå igang med en Use Case Workshop som Inception fasen oftest startes med i Unified Process (Larman, u.å.). Vores ønske var at arbejde med et ægte domæne for at kunne arbejde agilt, men vi blev nød til at tilgå det som et kunstigt domæne. Derfor valgte vi at ændre på processen og starte med at modellere problemdomænet med en koncept model. Det gjorde vi for at få et fælles grundlag for det (delvist) fiktive domæne. Normalt ville modelleringen komme efter at domænet er blevet afdækket med en PO i en use case workshop. I vores projekt brugte vi det som et værktøj til at blive enige om hvlilke antagelser vi ville lave om de manglende domæne information.

1.2 Opgaveformulering vs. agil metode

Det kunstige scenarie tvang os til at reflektere over, hvordan man arbejder agilt i en kontekst med stærkt begrænsede ressourcer og tid. Dilemmaet bestod i, om vi skulle lade projektets rammer — et to-ugers skoleprojekt — begrænse vores analyse fra start, eller om

vi skulle tilgå processen med et åbent sind og analysere domænet naivt, som om det var et fuldtskala-projekt.

Vi valgte at forblive tro mod den agile tankegang og gennemføre analysen uden forudindtagede begrænsninger. For at håndtere det begrænsede scope i praksis, inddrog vi det som en faktor i vores prioritering frem for i analysen. Konkret betød det, at use cases der lå uden for realistisk rækkevidde, blev tildelt en høj værdi på *Cost*-aksen i vores *value-driven* prioritering. Derved sikrede vi, at prioriterings-scoren naturligt sorterede urealistiske krav fra, samtidig med at vi opretholdt retten til pragmatisk at ekskludere use cases, der trods en favorabel score ikke kunne rummes inden for tidsrammen.

1.3 Iterationsstruktur

Vi tænker, at inception-fasen skal løbe over to dage (torsdag 12/3 + fredag 13/3)

Vi arbejder ud fra klients første domænebeskrivelse.

Vi diskuterer, om 1-3 iterationer er realistisk i projektet.

Inception oversigt

^1c3ba6

- OOA
- Domænemodel
- Krav-workshop - Use Case Model
- prioriter krav - ud fra risici og kundeværdi
- planlæg iterationer (Vi lægger en overordnet plan - men holder 'mandagsmøder' og 'opsamlingsmøder' i starten og slutningen af hver iteration med kommunikations-UML'er)

Vi diskuterer den agile proces i så kort et projekt. Vi tænker, at der skal være et par hele dagen til at lave færdige diagrammer og renskrive opgaven.

WPF er forholdsvist nyt for gruppen - det taget i betragtning tænker vi 3 iterationer på to dage hver er udgangspunktet

Dag 1 Elaboration starter mandag (16/3)

Vi diskuterer burnout-chart vs Gantt kort

1.4 Slices

2 Planlægning og tidsestimering

2.1 Use Case Workshop

2.2 Value- og risikodrevet

3 Forløbet

3.1 Faktiske iterationsstruktur

Den oprindelig plan bestod af tre iterationer, hvor hver iteration tog udgangspunkt i én use case opdelt i konkrete tasks. Det viste sig hurtigt, at vores første planlagte iteration (UC2: Se ledige transportmidler), indeholdt væsentligt mere kompleksitet end forventet, og ikke kunne gennemføres med en to-dages iteration.

Allerede på første dag af iteration 1 (mandag d. 16 marts) stod det klart, at inception-fasen ikke var tilstrækkeligt afsluttet. Derfor prioriterede vi at afsætte tid til at færdiggøre estimering og task-formulering frem for at påbegynde implementering på et uklart grundlag.

Vores opdeling i feature-orienterede tasks samt beslutningen om at arbejde med vertikale slices (Larman, u.å.) medførte mange arkitektoniske beslutninger blev truffet tidligt i forløbet.

Iteration 1 - UC 2 (16.-17. marts) (24 story points): Tasks:

De definerede tasks varierede både i kompleksitet og detaljegrad. Selvom de mest abstrakte tasks (f.eks. “Opret Bookingfunktion”) var frasorteret på dette tidspunkt, viste det sig at de resterende tasks dække over større tekniske beslutninger end forventet.

Vi prioriterede derfor de mest håndgribelige tasks: “Brugerdata” og “Håndter data for transportmidler”. Disse tasks, som oprindeligt var estimeret til 2 story points hver, kom reelt til at udgøre hovedparten af iterationen.

Implementeringen af bruger- og køretøjsdata dækkede reelt over et helt lag af beslutninger hveriblandt filformat, serialisering, repository-pattern og interfacedesign. Som følge heraf blev selve implementeringen implementeringen først påbegyndt d. 17. marts efter vi havde fastalgt en lagdelt arkitektur bestående af Domæne-, IO- og Services-lag.

Den efterfølgende task, “Se ledige køretøjer”, blev påbegyndt med henblik på at indrage præsentationslaget. Tasken var estimeret til 8 story points, men blev kun delvist implementeret inden for iterationen. Vi besluttede derfor at lade denne task fortsætte ind i næste iteration.

Iteration 2 - UC 2 (18.-24. marts) Den oprindelig plan for Iteration 2 var at arbejde med UC3 (Book transportmiddel). Vi vurderede dog,

at det var uhensigtsmæssigt, at implementere bookinglogik på et fundament vi ikke var tilfredse med. Derfor blev de tilbageværende tasks videreført til iteration 2.

I perioden onsdag d. 18 marts til fredag d. 20 marts blev tiden brugt på refactoring af arkitekturen. Dette arbejde var ikke direkte task-drevet og bidrog til yderligere forskydning i tidsplanen.

Refaktoringen resulterede dog i et mere generisk og genanvendeligt design. Eksempelvis kan en fremtidig BookingRepository implementeres ved at nedarve fra en FileHandler uden ændringer af eksisterende kode.

Mandag d. 23 marts blev WPF-delen færdiggjort for tasken, og der blev implementeret en AppFacade, for at skjule vores servicelag bag en samlet grænseflade samt oprettelse af DI-root'en.

Forløbet synliggjorde udfordringerne ved at følge UP i en begrænset tidsramme, særligt når der samtidig er fokus på implementering af design patterns og arkitektur generelt. På baggrund af dette besluttede vi, i slutningen af iteration 2, at reducere afvige af den planlagte proces og for at sikre implementeringen af designpattern.

4 Arkitektur og design

4.1 Composite Pattern

I vores første iteration af projektet var udgangspunktet use casen UC2 - 'Se ledige transportmidler for et bestemt tidspunkt'. For at kunne estimere implementeringen mere præcist, foldede vi den ud, så den blev flere mindre tasks. Vi oprettede domæneklasser for Car, Bike og Employee og dertilhørende json-filer. For at sikre en så fleksibel og vedligeholdelsesvenlig arkitektur som muligt, valgte vi at implementere et repository pattern. Formålet var at abstrahere dataadgangen, så applikationslaget ikke har direkte afhængighed til datakilden. Repositories kan virke overflødigt i vores indtil nu simple system, men gør det skalerbart til en fremtidig database som datakilde uden at skulle ændre logik i vores servicelag. Samtidig bidrager det til at holde vores domæneklasser rene og fri for datalagringslogik. Vi introducerede nu interfacet, `IRepository<T>`, fordi vi havde flere repositories med ens funktionalitet - de læser json-filerne og deserialiserer dem til objekter. For at undgå gentagelse oprettede vi den abstrakte FileHandler-klasse, som de tre konkrete repositories nedarver fra. Til at starte med implementerede vi en superklasse, BaseVehicleRepository, som CarRepository og BikeRepository nedarvede fra. Den blev introduceret for at kunne behandle forskellige transportmidler ens, da et Irepository Car ikke kunne anvendes som Irepository IVehicle. Med introduktionen af interfacet, IvehicleProvider, blev ansvaret flyttet væk fra superklassen, da det var interfacet, som VehicleService afhang af. Vores base-klasse gjorde det dog stadig muligt at genbruge logik mellem repositories.

Da vi senere indførte kovarians i `IRepository<T>` ved at bruge (out T) og ændre returtypen til `IEnumerable<T>` endte det med at skabe samme ønskede fleksibilitet, hvilket gjorde både IVehicleProvider og superklassen overflødige og muliggjorde, at de senere kunne fjernes.

Under implementeringen blev vi nemlig, med hjælp fra et genereret klassediagram, opmærksomme på den unødvendige kompleksitet i vores design, der gjorde ovennævnte klasse og interface overflødige. På dette tidspunkt bestod vores arkitektur stadig af to abstraktioner - IRepository T og IVehicleProvider, som servicelaget var afhængig af for at kunne håndtere forskellige transportmidler, da IRepository Car og IRepository Bike ikke kunne behandles som IRepository IVehicle. Klassediagrammet gjorde det tydeligt, at superklassen og IVehicle-

Provider kun eksisterede i systemet for at løse det problem. I et forsøg på at samle flere forskellige transportmidler under en struktur, implementerede vi klassen CompositeVehicleProvider, der samlede flere IVehicleProvider-instanser og fungerede som et composite pattern. Efter indplementeringen af kovarians blev det muligt at generalisere vores composite pattern og skabe `CompositeRepository<T>` som en generisk løsning. Med denne vigtige ændring blev designet åbent for nye samlinger af repositories i senere iterationer. Det betyder, at compileren nu tillader, at Car og Bike, samt alle fremtidige transportmidler, der implementerer IVehicle, kan behandles som IVehicle, og samtidig risikerer vi ikke, at vores employees bliver behandlet som transportmidler, fordi de ikke er en del af samme typehieraki.

Med indførelsen af kovarians kunne vi fjerne IVehicleProvider og BaseVehicleRepository og erstatte dem med en generisk `CompositeRepository<T>`. Det ender med at give os et generisk Composite Pattern, hvor servicelaget kan hente alle objekter af en type uden af kende til konkrete repositories.

4.2 Facade Pattern

5 Diskussion

5.1 Andre Patterns

5.2 SOLID og GRASP

6 Refleksion

6.1 Agil metode i et læringsscenarie

6.2 Task-formuleringer

7 Litteraturliste

Larman, C. (u.å.). *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development* (3rd ed). Prentice Hall PTR, c2005.

8 Bilag

Bilag A - SMS

Matias:

Det kommer jeg helt sikkert til. Tænker at vi kan snakke lidt om det på lørdag. Indtil da vil jeg gerne lige høre om det jeg hørte dig sige sidste gang er nogenlunde rigtigt: - Dig og dine kollegaer har daglige besøgslister på borgere i kommunen. - Til transport har i elcykler (nok til alle) og bil (til deling). - I skal selv beslutte rækkefølgen af jeres besøg. - I skal selv koordinere, kollegaer iblandt, hvem der bruger bil hvornår.

Lauras svar

Ja vi har alle 14 medarbejdere en borgerliste. Vi har 4 cykler, som vi deler med botilbuddet. Derudover har vi 4 biler til deling kun i Bostøtten. Nogle bruger også egen bil. Alle transportmidler bliver booket i Outlook. Der er nogle retningslinjer for hvor lang tid vi må booke frem mv. Vi koordinere selv hvornår borgerbesøg skal ligge. Men vi koordinere med hinanden på kryds og tværs. Da vi ofte deler en borger, så en har en borger i ulige uger og den anden i lige uger. Håber det giver mening

Bilag B - Use Case Model

ID	Navn	Beskrivelse
UC1	Se oversigt over bookinger	En medarbejder tilgår systemet og kan se oversigt over transport-bookinger.
UC2	Se ledige transportmidler for et bestemt tidspunkt	En medarbejder tilgår systemet, vælger et ønsket tidspunkt, og systemet viser ledige transportmidler.
UC3	Book transportmiddel	En medarbejder tilgår systemet og booker et ledigt transportmiddel i et givent tidspunkt.
UC4	Ændre booking	En medarbejder tilgår egen booking og redigerer bookingdetaljer.
UC5	Annullere booking	En medarbejder annullerer egen booking.
UC6	Registrer brug af privatbil	En medarbejder registrerer egen bil i systemet i et givent tidsrum.
UC7	Se hvilke medarbejdere der er på arbejde	En medarbejder kan se status på dagens booking-ejere.
UC8	Ajourfør medarbejder	Systemet modtager opdateret medarbejderliste fra Nexus.
UC9	Se borgerliste el. dagens besøg	En medarbejder kan se egen borgerliste og dagens besøg.
UC10	Planlæg rækkefølge af besøg	Systemet sammenligner besøgslister ud fra dato, borgerens adresse og koordinerer en optimal besøgsrækkefølge.
UC11	Overdrag borger til kollega	Systemet modtager en ajourført medarbejderliste, som indeholder en syg medarbejder. Systemet fordeler den syges borger til kollegaernes besøgslister.
UC12	Administrer transportmidler	Administrativ medarbejder åbner systemet og administrerer transportmidlerne.

UC2: Se ledige transportmidler for et bestemt tidspunkt

Use Case Section	**Comment**
Use Case Name	Se ledige transportmidler for et bestemt tidspunkt
Primary Actor	Specialpædagogerne
Stakeholders and Interests	**Pædagogerne:** vil gerne have en smertefri oplevelse med at koordinere brugen af transportmidler. **Bostedet:** interesse i at have et godt arbejdsmiljø, hvilket kan påvirkes af hvor nem arbejdsgangen er for deres medarbejdere. **Borgere:** interesseret i at deres støtte kommer til den aftalte tid, og at pædagogerne er fleksible i deres besøgstider.
Preconditions	Pædagogen har succesfuldt valgt et tidsrum.
Main Success Scenario	Pædagogen ser præcis hvilke biler og cykler der ikke er booket.
Extensions	**A:** tidsrum er ikke gyldigt, pædagogen ser en informativ fejlmeddelelse.
Special Requirements	Cross platform support

UC3: Book transportmiddel

Use Case Section	Comment
Use Case Name	Book transportmiddel
Primary Actor	Specialpædagog
Stakeholders and Interests	**Pædagogerne:** vil gerne have en smertefri oplevelse med at koordinere brugen af transportmidler. **Bostedet:** interesse i at have et godt arbejdsmiljø, hvilket kan påvirkes af hvor nem arbejdsgangen er for deres medarbejdere. **Borgere:** interesseret i at deres støtte kommer til den aftalte tid, og at pædagogerne er fleksible i deres besøgstider.
Preconditions	Valgt gyldig tidsrum. (Se UC2)
Success Guarantee	Transportmidlet er ikke længere tilgængelig i det valgte tidsrum og det står som booket i medarbejderens navn.
Main Success Scenario	**1:** Ser ledige transportmidler **2:** Vælger ledig transportmiddel **3:** Booker valgte transportmiddel **4:** Modtager bekræftigelse
Extensions	**A:** Vælger transportmiddel der allerede er booket. **1:** Ser ledige transportmidler **2:** Vælger ledig transportmiddel **3:** Prøver at booker valgte transportmiddel **4:** Modtager en fejlmeddelelse **5:** Sender fejlen til admin
Special Requirements	**1:** Cross platform support **2:** Håndterer edge cases hvor dobbelt booking kan forekomme